



heber amaral <heberfa@gmail.com>

LV Simpósio Brasileiro de Pesquisa Operacional | Trabalho submetido

2 mensagens

SBPO 2023 <notificacoes@galoamail.com.br>
Responder a: SBPO 2023 <sbpo@sobrapo.org.br>
Para: heberfa@gmail.com

11 de junho de 2023 às 19:59

LV Simpósio Brasileiro de Pesquisa Operacional

**LV Simpósio Brasileiro de Pesquisa Operacional | Trabalho submetido | LV Simpósio Brasileiro de Pesquisa Operacional**

Prezado(a) Heber Amaral,

Recebemos seu trabalho intitulado "COMPARAÇÃO DA ESTRATÉGIA DO APRIMORAMENTO ADIADO COM AS ESTRATÉGIAS DO MELHOR APRIMORANTE E DO PRIMEIRO APRIMORANTE EM BUSCAS LOCAIS APLICADO AO PROBLEMA DO CAIXEIRO VIAJANTE" para ser avaliado no eixo 14. OC – Otimização Combinatória do LV Simpósio Brasileiro de Pesquisa Operacional.

Você poderá acompanhar o status da avaliação em sua área de usuário, e assim que for concluída você receberá um email informando o resultado.

Atenciosamente,

Comitê Científico | LV Simpósio Brasileiro de Pesquisa Operacional

Este email foi enviado para heberfa@gmail.com.

Por que recebi este email?

LV Simpósio Brasileiro de Pesquisa Operacional usa Galoá

[Saiba mais sobre o Galoá](#)

SBPO 2023 <notificacoes@galoamail.com.br>
Responder a: SBPO 2023 <sbpo@sobrapo.org.br>
Para: heberfa@gmail.com

11 de junho de 2023 às 20:20

LV Simpósio Brasileiro de Pesquisa Operacional



LV Simpósio Brasileiro de Pesquisa Operacional | Trabalho submetido | LV Simpósio Brasileiro de Pesquisa Operacional

Prezado(a) Heber Amaral,

Recebemos seu trabalho intitulado "Estratégia do Aprimoramento adiado para buscas locais: Usando características de ótimos locais como critério na seleção de soluções aprimorantes" para ser avaliado no eixo 14. OC – Otimização Combinatória do LV Simpósio Brasileiro de Pesquisa Operacional.

Você poderá acompanhar o status da avaliação em sua área de usuário, e assim que for concluída você receberá um email informando o resultado.

Atenciosamente,

Comitê Científico | LV Simpósio Brasileiro de Pesquisa Operacional

Este email foi enviado para heberfa@gmail.com.

Por que recebi este email?

LV Simpósio Brasileiro de Pesquisa Operacional usa Galoá

Saiba mais sobre o Galoá

COMPARAÇÃO DA ESTRATÉGIA DO APRIMORAMENTO ADIADO COM AS ESTRATÉGIAS DO MELHOR APRIMORANTE E DO PRIMEIRO APRIMORANTE EM BUSCAS LOCAIS APLICADO AO PROBLEMA DO CAIXEIRO VIAJANTE

RESUMO

A busca local é uma ferramenta fundamental no desenvolvimento de algoritmos heurísticos. Neste trabalho comparamos a estratégia do aprimoramento adiado para busca local com estratégias de busca local clássicas. A estratégia do aprimoramento adiado usa desigualdades comumente usadas como cortes de otimização no contexto da programação linear inteira. O método foi implementado e avaliado usando o problema do caixeiro viajante com a vizinhança 2-opt como caso de teste. Os resultados computacionais mostram que a estratégia do aprimoramento adiado, embora mais lenta, obtém melhores ótimos locais em comparação com as estratégias tradicionais de busca local. A comparação dos resultados é favorável à estratégia do aprimoramento adiado em experimentos com tempo de computação fixo ou com alvo fixo.

PALAVRAS CHAVE. Otimização combinatória, Busca local, Heurística, 2-Opt, Problema do Caixeiro viajante

ABSTRACT

Local search is a fundamental tool in the development of heuristic algorithms. In this work we compare the delayed improvement strategy for local search with others well know local search strategies. The delayed improvement local search strategy uses inequalities commonly used as cuts in the context of integer linear programming. The method was implemented and evaluated using the traveling salesman problem with the 2-opt neighborhood as a test case. Computational results show that the delayed enhancement strategy, although slower, obtains better local optima compared to traditional local search strategies. The comparison of results favors the strategy of delayed improvement local search in experiments with fixed computation time or with a fixed target.

KEYWORDS. Combinatorial optimization, Heuristic, 2-Opt, Traveling salesman problem

1. Introdução

Encontrar a solução ótima para uma instância específica de um problema de otimização combinatória pode ser uma tarefa computacionalmente desafiadora. Os métodos utilizados para resolver problemas complexos desse tipo geralmente podem ser agrupados em duas categorias principais: métodos exatos, que visam encontrar a solução ótima do problema e métodos heurísticos, que procuram encontrar uma solução suficientemente boa dentro de um prazo razoável. Este artigo concentra-se em uma das técnicas heurísticas mais reconhecidas, a busca local.

Existem diversas abordagens para implementar a busca local, cada uma diferindo na maneira como a nova solução é escolhida entre as soluções aprimorantes disponíveis na vizinhança atual. Uma dessas estratégias é a Estratégia do Aprimoramento Adiado (do inglês, Delayed Improvement Local Search, DILS) [Amaral et al., 2021]. Essa estratégia é implementada através da identificação dos atributos comuns presentes em todos os ótimos locais e, a cada iteração da busca local, selecionando um vizinho aprimorante que possua o menor número possível de atributos em comum com os ótimos locais. Com isso a estratégia busca evitar ótimos locais de baixa qualidade. Neste trabalho o desempenho do DILS é comparado com as estratégias conhecidas de Melhor Aprimorante (do inglês, Best Improvement Local Search, BILS) e Primeiro Aprimorante (do inglês, First Improvement Local Search, FILS). Para comparar os métodos tradicionais com o DILS, consideramos o conhecido problema do caixeiro viajante (TSP) com a vizinhança 2-opt [Johnson e McGeoch, 1997].

O restante deste trabalho está organizado da seguinte forma: A seção 2 fornece material de referência e uma revisão da literatura relacionada. Na Seção 3 explicamos como nossos experimentos computacionais são estruturados. Os resultados computacionais se concentram em demonstrar o comportamento do DILS, bem como avaliar o desempenho. A seção 4 apresenta resultados para o TSP. Finalmente, as considerações finais seguem na Seção 5.

2. Revisão de Literatura

Considere uma solução s de um problema de otimização combinatória. Seja $f(s)$ o custo da solução s . Seja N um operador de vizinhança que, a partir de uma solução dada, gera um conjunto de soluções modificadas. Soluções melhores que s podem estar disponíveis em $N(s)$ em uma determinada iteração da pesquisa. As estratégias mais conhecidas para busca local diferem na seleção do vizinho melhorado, que muitas vezes orienta a busca local para ótimos locais de qualidade diferente.

A estratégia BILS seleciona a cada iteração o vizinho com o melhor valor da função objetivo. A estratégia FILS seleciona a cada iteração o primeiro vizinho avaliado que é melhor que a solução atual. Uma dessas duas estratégias é implementada na grande maioria da literatura de busca local e metaheurística. Hansen e Mladenović [2006] analisou e comparou computacionalmente essas duas estratégias junto com a pior melhoria (seleção do vizinho menos melhorado) no contexto de 2-opt para o TSP. Eles concluíram que o FILS é o melhor método quando a busca parte de uma solução inicial aleatória, enquanto o BILS é melhor quando a solução inicial é construída com uma heurística gulosa.

Várias técnicas foram introduzidas ao longo dos anos para melhorar a eficácia da busca local e evitar que ela fique presa em ótimos locais de baixa qualidade. Isso inclui metaheurísticas como *simulated annealing* [Kirkpatrick et al., 1983; Černý, 1985], busca local guiada [Voudouris, 1997], busca tabu [Glover, 1986, 1989, 1990], busca de vizinhança variável [Mladenović e Hansen, 1997] e busca local iterada [Lourenço et al., 2010]. Alves et al. [2022] apresenta um estudo comparativo de cinco metaheurísticas aplicadas ao problema do caixeiro viajante múltiplo (mTSP), sendo

elas: Greedy Randomized Adaptive Search Procedure (GRASP), Biased Random-Key Genetic Algorithm (BRKGA), Large Neighborhood Search (LNS), Variable Neighborhood Search (VNS) e Iterated Local Search (ILS). Os experimentos de Alves et al. [2022] mostraram que o BRKGA foi a metaheurística que obteve os melhores resultados para o mTSP.

Quando usado no contexto de uma metaheurística, não apenas a qualidade da solução retornada é importante, mas também a velocidade de execução da busca como um todo. O *FILS* foca na velocidade avaliando o menor número possível de vizinhos da vizinhança atual. Por outro lado, o *BILS* se concentra em diminuir o número de iterações da busca, melhorando a função objetivo o máximo possível. A literatura discute diversas técnicas para acelerar uma avaliação de vizinhança, como avaliações delta (a avaliação de um vizinho baseada na diferença de custo em relação à solução atual) [Bertsimas, 1988], avaliações heurísticas (uma avaliação aproximada do custo função em vez de sua avaliação exata) [Prandtstetter e Raidl, 2008; Solnon et al., 2008], listas de candidatos (avaliando primeiro os vizinhos que são considerados com maior probabilidade de melhorar) [Pekny e Miller, 1994] e reduções de vizinhança (a priori excluindo vizinhos que podem ser comprovados como não melhorar) [Nowicki e Smutnicki, 1996].

A abordagem *DILS*, apreendida por Amaral et al. [2021], a ser usada neste artigo, decide de forma menos gulosa qual vizinho selecionar como a próxima solução atual em cada iteração. Ele move a busca para um vizinho melhor que tem o maior potencial para novas melhorias ao aplicar o mesmo operador de vizinhança para fazer movimentos subsequentes. A nova abordagem depende da disponibilidade de rapidez para executar testes de otimização local. Lancia et al. [2015] introduziu um conjunto de testes para a vizinhança 2-opt do TSP e para outros problemas de otimização combinatória.

Para entender melhor considere $A = \{a_1, a_2, \dots, a_m\}$ um conjunto de atributos presente em ótimos locais da vizinhança N e seja $P = \{p_1, p_2, \dots, p_t\}$ um conjunto de verificação de precondições de optimalidade de N . Definimos $A_i \subseteq A$ um subconjunto de atributos que são relevantes para a precondição p_i . Seja $k_P(s) = |\{(i, j) : p_i(a_j(s)) = \text{false}, 1 \leq i \leq t, a_j \in A_i\}|$ o número de verificações em P avaliadas como falsas para a solução s . Por definição, $k_P(s) = 0$ para cada ótimo local s da vizinhança N .

O *DILS* seleciona a cada iteração uma solução s' que maximiza $k(s')$ entre todas as soluções aprimorantes de $N(s)$. O custo da solução é usado como desempate. O algoritmo 2.1 descreve a abordagem proposta.

Algorithm 2.1: Delayed improvement local search. [Amaral et al., 2021]

```

1 Delayed Improvement Local Search(s)
2 while  $I(s) = \{s' \in N(s) : f(s') < f(s)\}$  não for vazio do
3    $k \leftarrow \max_{s' \in I(s)} k_P(s')$ ;
4    $s'' \leftarrow \operatorname{argmin}_{s' \in I(s), k_P(s')=k} f(s')$ ;
5    $s \leftarrow s''$ ;
6 retorna s;
```

A ideia por trás do método é melhorar a solução atual evitando, tanto quanto possível, a convergência para um ótimo local de baixa qualidade. Ou seja, a busca deve ter mais oportunidades de encontrar soluções de alta qualidade por evitar ótimos locais de má qualidade.

A desvantagem da abordagem *DILS* é em termos de tempo de execução. Como todas as verificações precisam ser avaliadas para todas as soluções candidatas, cada iteração da pesquisa é

computacionalmente mais exigente do que no *BILS* e *FILS*.

É crucial enfatizar que a abordagem *DILS*, em contraste com estratégias como a Tabu Search e outras metaheurísticas, não realiza a acumulação de informações dos ótimos locais ao longo de sua execução. O que verdadeiramente distingue a abordagem *DILS* das abordagens *BILS* e *FILS* é a sua consideração não apenas do delta de custo, mas também do delta de pré-condições violadas. Essa consideração envolve a avaliação das desigualdades (inequações) violadas em uma rota, como as proposta por Lancia et al. [2015], o que é uma medida prática do número de vezes que tais violações ocorrem na rota. Dado que o movimento *2-OPT* modifica apenas dois vértices a cada iteração, o cálculo do delta de pré-condições de otimalidade violadas leva em conta a influência desses vértices na alteração do número de pré-condições violadas. Isso é análogo ao método empregado no cálculo do delta de custo, onde a diferença de custos entre os vértices removidos e adicionados também é considerada.

Tanto o *BILS* quanto o *FILS* selecionam a cada iteração a solução avaliada com o melhor custo. Após qualquer iteração a solução atual é a melhor já avaliada desde o início da busca. Esta propriedade não vale para o *DILS* e, conseqüentemente, pode acontecer que a solução retornada ao final da busca não seja a melhor já avaliada: um dos vizinhos avaliados em uma etapa anterior pode ter sido melhor que o ótimo local final. Embora esse comportamento seja teoricamente possível, Amaral et al. [2021] relatam que é extremamente raro na prática e, portanto, não vale a pena gastar recursos computacionais para armazenar quaisquer soluções adicionais encontradas durante a pesquisa local.

3. Configuração dos testes

Para investigar o comportamentos metodos, executamos *DILS*, *BILS* e *FILS* em uma única instância de teste começando com a mesma solução inicial. Nesses testes, calculamos o número de verificadores não satisfeitos em cada iteração para todos os três métodos. Isso nos permite mostrar como os três métodos diferem em termos de suas progressões de qualidade de solução e o número de verificadores violados em cada iteração da busca. Além disso, para investigar o comportamento, analisamos o número de iterações realizadas para atingir um ótimo local por cada um dos métodos.

Ao analisar o desempenho do *DILS*, consideramos os métodos de busca local em uma estrutura multi-início. Cada método é reiniciado ao atingir um ótimo local, enquanto registra o valor do melhor ótimo local encontrado. O consumo de tempo do *DILS* pode ser maior do que o consumo de tempo dos outros métodos devido ao maior esforço computacional por iteração ao tentar executar mais iterações antes de convergir para um ótimo local. Portanto, ao avaliar o desempenho do *DILS*, é injusto executar cada um dos métodos para qualquer número fixo de reinicializações.

Em vez disso, consideramos duas configurações diferentes para fazer comparações justas. Na primeira configuração, fixamos um limite de tempo de execução e executamos repetidamente uma determinada pesquisa local (*BILS*, *FILS* ou *DILS*) a partir de diferentes soluções iniciais até que o limite de tempo seja atingido. A melhor solução encontrada é retornada. Esta abordagem multi-start é uma comparação mais justa dos métodos, uma vez que os métodos mais rápidos podem ser repetidos mais vezes do que os mais lentos, aumentando assim sua probabilidade de encontrar um bom ótimo local.

Na segunda configuração, consideramos instâncias individuais e produzimos gráficos *Time to Target* (TTT) [Aiex et al., 2007]. Para criar esses gráficos, definimos um custo de solução alvo e medimos o tempo necessário (até um limite de tempo geral alto) para cada método alcançar uma solução com um valor de função objetivo pelo menos tão bom quanto o alvo. Repetimos o experimento um grande número de vezes e classificamos o tempo para atingir os valores em ordem

crescente. Plotando esses dados obtemos uma estimativa da probabilidade $p(t)$ do algoritmo obter uma solução com a qualidade desejada em função do tempo de execução.

Para os testes consideramos 30 instâncias do TSPLib [Reinelt, 1991].

3.1. Heurística construtiva

Os procedimentos de busca local partem de uma solução inicial. Neste trabalho usamos dois métodos diferentes para obter essas soluções iniciais para TSP. O método principal é uma heurística de inserção. Ele começa com uma rota que consiste em um vértice selecionado aleatoriamente e o vértice mais próximo a ele. Então, durante $n - 2$ iterações, um vértice aleatório que ainda não está na rota é inserido na posição que aumenta menos o comprimento da rota. A outra abordagem constrói uma solução inicial criando uma permutação aleatória dos vértices. Em geral, o método principal produz soluções muito melhores do que a permutação aleatória.

3.2. Configuração computacional

Os métodos de busca local foram implementados usando a linguagem de programação C. O código foi compilado com o compilador gcc usando o sinalizador de otimização O3. Os experimentos foram executados em uma máquina Intel(R) Xeon(R) E5405 de 64 bits, 2 GHz com 4 processadores e 16,0 GB de RAM executando o sistema operacional Ubuntu 18.04.3 LTS. Para gerar os gráficos, por exigir muitas execuções com diferentes alvos, usamos máquinas Intel(R) Xeon(R) de 64 bits, 3.1 GHz com 4 processadores e 16,0 GB de RAM executando o sistema operacional Debian GNU/Linux 11. Os métodos foram todos programados de acordo com os mesmos padrões de programação e com o mesmo nível de otimização.

4. Resultados computacionais para o TSP

Os resultados computacionais são conduzidos em instâncias do TSP e os resultados obtidos com a abordagem *DILS* são comparados ao desempenho correspondente das abordagens *BILS* e *FILS*.

Na tabela 1 mostramos as abordagens sendo executadas 10 vezes com soluções iniciais aleatórias por 300 segundos cada. Já na tabela 2 mostramos as abordagens sendo executadas 10 vezes com soluções iniciais geradas pela heurística construtiva por 300 segundos cada. Em ambas as tabelas mostramos o tempo médio de execução, o número de execuções (reinícios), o custo médio obtido, o custo mínimo obtido (melhor caso), o custo máximo obtido (pior caso), o ótimo global retirado da literatura, o erro em relação a média e o erro em relação ao melhor resultado obtido.

Pode-se observar na tabela 1, onde os melhores valores estão destacados em negrito, que a abordagem *DILS* tem um desempenho pior se comparada às outras abordagens quando executado a partir de soluções iniciais aleatórias. Já na tabela 2, que mostra resultados obtidos a partir de soluções iniciais geradas por heurística construtiva, a abordagem *DILS* obteve todos os ótimos globais das instâncias testadas, enquanto a abordagem *BILS* obteve 9 ótimos globais das 11 instâncias testadas e a abordagem *FILS* obteve 7 ótimos globais das 11 instâncias testadas.

Uma solução aleatória tem, em geral, um número maior de vizinhos de melhoria do que uma solução obtida por uma heurística construtiva. Assim, nas primeiras iterações, o *DILS* é capaz de encontrar vizinhos de melhoria que aumentam o número de verificadores não satisfeitos, após o que converge muito lentamente para um ótimo local. As grandes diferenças no número de iterações realizadas antes que um ótimo local seja alcançado, mais o custo extra para avaliar os verificadores, torna improvável que o *DILS* possa compensar a diferença no tempo computacional pela qualidade da solução obtida. Em consequência, concluímos que o *DILS* não é competitivo quando a solução inicial é gerada aleatoriamente, e o restante dos experimentos considera soluções iniciais obtidas com as heurísticas de construção.

Instância	2-OPT Melhor Ap.					Ótimo	E. Média	E. Melhor
	T. Médio	Exe.	C. Médio	C. Mín.	C. Máx.			
att48	300,00	127651,60	10628,00	10628	10628	10628	0,00%	0,00%
berlin52	300,00	101865,80	7542,00	7542	7542	7542	0,00%	0,00%
kroA100	300,01	12256,00	21287,40	21282	21296	21282	0,03%	0,00%
kroA150	300,04	3578,30	26859,30	26596	27037	26524	1,26%	0,27%
kroB150	300,04	3567,80	26439,10	26195	26581	26130	1,18%	0,25%
kroC100	300,01	12274,30	20754,70	20749	20798	20749	0,03%	0,00%
kroD100	300,01	12521,60	21356,50	21294	21450	21294	0,29%	0,00%
kroE100	300,01	12563,30	22106,40	22068	22162	22068	0,17%	0,00%
lin105	300,01	10368,20	14379,00	14379	14379	14379	0,00%	0,00%
pr107	300,02	9995,40	44365,90	44303	44442	44303	0,14%	0,00%
pr124	300,03	5913,50	59030,00	59030	59030	59030	0,00%	0,00%
Erro Médio							0,59%	0,37%
2-OPT Primeiro Ap.								
	T. Médio	Exe.	C. Média	C. Mín.	C. Máx.			
att48	300,00	130093,80	10628,00	10628	10628	10628	0,00%	0,00%
berlin52	300,00	111704,20	7542,00	7542	7542	7542	0,00%	0,00%
kroA100	300,01	10586,90	21282,00	21282	21282	21282	0,00%	0,00%
kroA150	300,04	2746,70	26759,10	26653	26836	26524	0,89%	0,49%
kroB150	300,06	2758,30	26337,20	26273	26421	26130	0,79%	0,55%
kroC100	300,01	10472,90	20749,00	20749	20749	20749	0,00%	0,00%
kroD100	300,01	10725,00	21408,60	21307	21477	21294	0,54%	0,06%
kroE100	300,02	10856,50	22143,00	22073	22185	22068	0,34%	0,02%
lin105	300,01	9067,70	14386,70	14379	14412	14379	0,05%	0,00%
pr107	300,01	10172,40	44461,20	44347	44526	44303	0,36%	0,10%
pr124	300,03	5660,10	59042,40	59030	59092	59030	0,02%	0,00%
Erro Médio							0,58%	0,43%
DILS								
	T. Médio	Exe.	C. Média	C. Mín.	C. Máx.			
att48	300,54	221,60	10641,00	10628	10658	10628	0,12%	0,00%
berlin52	301,06	154,90	7550,50	7542	7627	7542	0,11%	0,00%
kroA100	312,90	11,70	21601,40	21381	21925	21282	1,50%	0,47%
kroA150	373,20	3,00	27231,20	26737	27520	26524	2,67%	0,80%
kroB150	368,21	3,00	27133,80	26667	27854	26130	3,84%	2,06%
kroC100	314,03	12,10	21060,10	20901	21151	20749	1,50%	0,73%
kroD100	318,01	12,00	21714,00	21466	21977	21294	1,97%	0,81%
kroE100	314,08	12,30	22402,20	22111	22688	22068	1,51%	0,19%
lin105	311,09	9,10	14535,80	14502	14605	14379	1,09%	0,86%
pr107	316,33	9,00	46177,80	45326	46673	44303	4,23%	2,31%
pr124	327,01	5,00	60184,70	59087	61133	59030	1,96%	0,10%
Erro Médio							2,04%	1,02%

Instância	2-OPT Melhor Ap.					Ótimo	E. Média	E. Melhor
	T. Médio	Exe.	C. Médio	C. Mín.	C. Máx.			
att48	300,00	646141,70	10628,00	10628	10628	10628	0,00%	0,00%
berlin52	300,00	576040,90	7542,00	7542	7542	7542	0,00%	0,00%
kroA100	300,00	93306,40	21284,10	21282	21296	21282	0,01%	0,00%
kroA150	300,00	33037,60	26695,30	26596	27037	26524	0,65%	0,27%
kroB150	300,01	31845,40	26215,90	26165	26581	26130	0,33%	0,13%
kroC100	300,00	93336,80	20753,30	20749	20798	20749	0,02%	0,00%
kroD100	300,00	99233,90	21314,60	21294	21344	21294	0,10%	0,00%
kroE100	300,00	99064,70	22090,70	22068	22111	22068	0,10%	0,00%
lin105	300,00	127074,40	14379,00	14379	14379	14379	0,00%	0,00%
pr107	300,00	160115,20	44303,00	44303	44303	44303	0,00%	0,00%
pr124	300,00	111683,00	59030,00	59030	59030	59030	0,00%	0,00%
Erro Médio							0,11%	0,04%
2-OPT Primeiro Ap.								
	T. Médio	Exe.	C. Média	C. Mín.	C. Máx.			
att48	300,00	1034250,20	10628,00	10628	10628	10628	0,00%	0,00%
berlin52	300,00	839930,80	7542,00	7542	7542	7542	0,00%	0,00%
kroA100	300,00	132670,70	21282,00	21282	21282	21282	0,00%	0,00%
kroA150	300,00	44406,20	26695,70	26637	26797	26524	0,65%	0,43%
kroB150	300,00	44073,60	26279,90	26231	26347	26130	0,57%	0,39%
kroC100	300,00	131138,60	20749,00	20749	20749	20749	0,00%	0,00%
kroD100	300,00	142148,30	21325,80	21307	21358	21294	0,15%	0,06%
kroE100	300,00	139057,50	22094,00	22073	22120	22068	0,12%	0,02%
lin105	300,00	180728,80	14379,00	14379	14379	14379	0,00%	0,00%
pr107	300,00	227934,60	44303,00	44303	44303	44303	0,00%	0,00%
pr124	300,00	147996,00	59030,00	59030	59030	59030	0,00%	0,00%
Erro Médio							0,14%	0,08%
DILS								
	T. Médio	Exe.	C. Média	C. Mín.	C. Máx.			
att48	300,00	52471,60	10628,00	10628	10628	10628	0,00%	0,00%
berlin52	300,01	50713,80	7542,00	7542	7542	7542	0,00%	0,00%
kroA100	300,02	9267,20	21282,00	21282	21282	21282	0,00%	0,00%
kroA150	300,07	3241,50	26735,10	26524	26825	26524	0,80%	0,00%
kroB150	300,06	3282,90	26259,90	26130	26344	26130	0,50%	0,00%
kroC100	300,01	9005,30	20749,00	20749	20749	20749	0,00%	0,00%
kroD100	300,01	9360,60	21386,50	21294	21423	21294	0,43%	0,00%
kroE100	300,02	8862,30	22092,40	22068	22121	22068	0,11%	0,00%
lin105	300,01	16034,40	14383,40	14379	14401	14379	0,03%	0,00%
pr107	300,01	27381,30	44303,00	44303	44303	44303	0,00%	0,00%
pr124	300,01	18547,00	59030,00	59030	59030	59030	0,00%	0,00%
Erro Médio							0,17%	0,00%

Na tabela 3, mostramos resultados com limite de tempo de execução de 300 segundos em uma estrutura *multi-start*. Cada instância é resolvida dez vezes por DILS, BILS e FILS. Como observado, o DILS requer mais iterações para atingir um ótimo local. Nos experimentos com limite de tempo fixo, o número de reinícios executados por DILS é uma ordem de grandeza menor que o número de reinícios por BILS e FILS.

Os resultados mostrados aqui confirmam o que está na Tabela 3 onde é relatado tanto o custo médio das soluções encontradas nas dez execuções (Avg. Z) quanto o custo da melhor solução nas dez execuções (Min. Z). Quando consideramos o custo mínimo nas dez execuções para cada instância, o DILS obteve os melhores resultados para 25 instâncias, enquanto o BILS e o FILS encontraram as melhores soluções para apenas 10 instâncias cada. Aplicando um teste qui-quadrado, rejeita-se a hipótese de que o desempenho do DILS é igual ao desempenho do BILS, com um valor P de 6×10^{-9} . A mesma conclusão se aplica ao comparar DILS com FILS. Ao considerar os custos médios das 10 execuções, o número de melhores desempenhos é 17 para DILS, 12 para BILS e 9 para FILS. Aplicando novamente um teste qui-quadrado, a hipótese de que o desempenho médio do DILS é igual ao desempenho médio do FILS é rejeitada com um valor-P de 0,001, enquanto o mesmo teste falha ao rejeitar a hipótese de que DILS e BILS poderiam ter o mesmo desempenho médio do valor-P é 0,06.

No teste seguinte avaliamos o desempenho usando gráficos *Time to Target* (TTT) [Aiex et al., 2007]. Usamos um limite de tempo total de 600 segundos e realizamos 100 execuções que terminam ao atingir o alvo escolhido ou ao atingir o limite de tempo. Aqui mostraremos os resultados para a instância pr439. O alvo mais difícil é definido para 1% acima do melhor custo médio de solução sobre os três algoritmos na Tabela 3 arredondado para o inteiro mais próximo. Os outros três alvos estão 2% e 3% acima do alvo mais difícil, arredondado para o inteiro mais próximo.

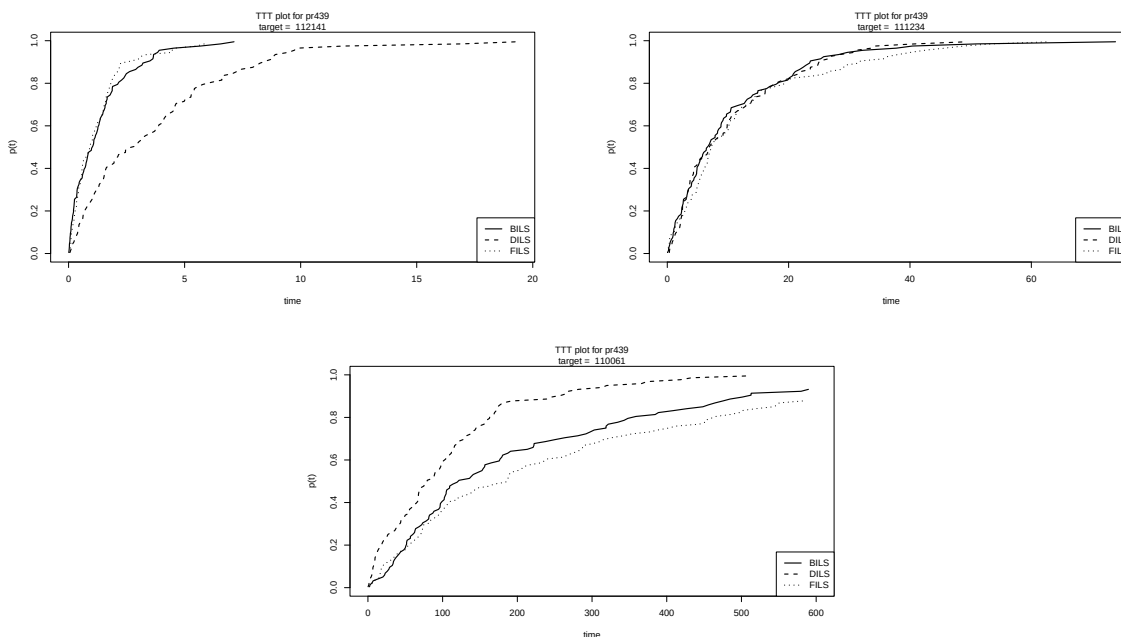


Figura 1: Gráficos TTT para a instância pr439, com o alvo mais fácil no canto superior esquerdo e o alvo mais difícil no centro inferior

Os gráficos mostram que, quando o alvo é fácil, *BILS* e *FILS* têm melhor ou semelhante ao

Tabela 3: Resultados para DILS, BILS e FILS. Executando por 300 segundos, 10 repetições, partindo da solução gerada a partir da heurística construtiva. fonte:[Amaral et al., 2021]

Instance	BILS		FILS		DILS	
	Avg. Z	Min. Z	Avg. Z	Min. Z	Avg. Z	Min. Z
1	berlin52	7542.0	7542	7542.0	7542	7542
2	ch130	6117.6	6110	6117.0	6110	6110
3	ch150	6577.3	6561	6566.0	6548	6548
4	d1291	55200.8	54935	55401.4	54961	55287.6
5	kroA200	29678.0	29573	29686.2	29599	29683.3
6	kroB200	29728.2	29645	29733.3	29632	29793.0
7	kroC100	20749.0	20749	20749.0	20749	20749
8	kroD100	21294.0	21294	21296.8	21294	21308.4
9	lin105	14379.0	14379	14379.0	14379	14379
10	linhp318	43245.2	43103	43255.2	43059	43245.9
11	p654	35049.4	35007	35069.7	35049	35069.4
12	pcb1173	62440.5	61885	62175.9	61866	61765.1
13	pcb442	53506.1	53383	53495.1	53117	53038.3
14	pr1002	275906.2	274720	276056.4	274975	275679.4
15	pr152	73682.0	73682	73682.0	73682	73682
16	pr2392	416929.7	414049	415425.0	412988	415618.1
17	pr299	49185.2	48943	49179.5	49011	49220.8
18	pr439	110308.6	109730	110489.4	109937	110048.8
19	rat195	2405.2	2387	2402.2	2380	2399.8
20	rd400	15908.6	15856	15904.4	15835	15875.7
21	rl1323	290386.3	285853	291240.3	287928	289982.0
22	rl1889	346631.9	343891	348353.9	346499	344231.4
23	tsp225	4036.9	4031	4041.2	4022	4037.5
24	u1060	238396.8	237519	238504.0	237602	237519.0
25	u1432	166572.3	165792	166646.2	166054	166621.3
26	u1817	64391.0	64083	64514.0	64137	64380.7
27	u2152	73100.0	72417	73213.5	72675	72780.1
28	u2319	248396.3	247856	248218.4	247735	248246.1
29	u574	38865.3	38745	38821.6	38597	38807.8
30	vm1748	363912.0	361293	363577.3	362488	362627.3
No. of best		12	10	9	10	17
Avg. % gap		0.69	0.27	0.71	0.30	0.53
Std. % gap		0.58	0.37	0.61	0.45	0.37

desempenho do que *DILS*. No entanto, todos os três algoritmos atingem o alvo em poucos segundos. Por outro lado, quando o objetivo fica mais difícil de atingir, o desempenho do *DILS* melhora em relação ao *BILS* e *FILS*. Com o alvo mais difícil, o desempenho do *DILS* é superior. Podemos concluir que enquanto o *DILS* é mais lento do que as estratégias clássicas para obter soluções de qualidade média, no longo prazo ele tende a alcançar soluções de melhor qualidade.

5. Conclusões

Neste trabalho, comparamos a estratégia do aprimoramento adiado (*DILS*), uma técnica de busca local baseada na ideia de selecionar como próxima solução o vizinho melhorado que possui menos características de ótimos locais, com as abordagens clássicas, Melhor Aprimorante e Primeiro aprimorante.

Implementamos os métodos para a busca local 2-opt para o problema do caixeiro viajante (*TSP*). Os resultados computacionais mostram que, para as instâncias testadas, o *DILS* tem um desempenho melhor do que as implementações clássicas de busca local, melhor aprimorante e busca local de primeiro aprimorante, quando a solução inicial é construída com heurísticas de construção gulosa.

Pesquisas futuras incluem a compara de *DILS* a outras vizinhanças para o *TSP*, como o *node swap*, para o qual Lancia et al. [2015] também fornece desigualdades de busca local e 3-opt. O *DILS* também pode ser aplicado a outros problemas de otimização combinatória, além do *TSP*. Pode-se implementar, a ideia do *DILS* em metaheurísticas baseadas em busca local, como busca tabu, busca local iterada e busca de vizinhança variável. Em cada um deles, pode-se verificar se adiar a convergência para um ótimo local pode ser benéfico.

Referências

- Aiex, R. M., Resende, M. G., e Ribeiro, C. C. (2007). TTT plots: a perl program to create time-to-target plots. *Optimization Letters*, 1(4):355–366.
- Alvesa, D. J. F., Avelino, B., de Faria, V. C. N., Macedo, E. A., Martini, V. G., Vieira, B. S., e Chaves, A. A. (2022). Estudo comparativo de metaheurísticas aplicadas ao problema do caixeiro viajante múltiplo. *54 SIMPÓSIO BRASILEIRO DE PESQUISA OPERACIONAL*.
- Amaral, H. F., Urrutia, S., e Hvattum, L. M. (2021). Delayed improvement local search. *Journal of Heuristics*, 27:923–950.
- Bertsimas, D. (1988). *Probabilistic combinatorial optimization problems*. PhD thesis, Massachusetts Institute of Technology.
- Černý, V. (1985). Thermodynamical approach to the traveling salesman problem: An efficient simulation algorithm. *Journal of Optimization Theory and Applications*, 45(1):41–51.
- Glover, F. (1986). Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5):533–549.
- Glover, F. (1989). Tabu search—part i. *ORSA Journal on Computing*, 1(3):190–206.
- Glover, F. (1990). Tabu search—part ii. *ORSA Journal on Computing*, 2(1):4–32.
- Hansen, P. e Mladenović, N. (2006). First vs. best improvement: an empirical study. *Discrete Applied Mathematics*, 154(5):802–817.

- Johnson, D. S. e McGeoch, L. A. (1997). The traveling salesman problem: A case study in local optimization. In Aarts, E. H. L. e Lenstra, J. K., editors, *Local Search in Combinatorial Optimization*, p. 215–310. John Wiley and Sons, Chichester, United Kingdom.
- Kirkpatrick, S., Gelatt, C. D., e Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598):671–680.
- Lancia, G., Rinaldi, F., e Serafini, P. (2015). Local search inequalities. *Discrete Optimization*, 16: 76–89.
- Lourenço, H. R., Martin, O. C., e Stützle, T. (2010). Iterated local search: Framework and applications. In Gendreau, M. e Potvin, J.-Y., editors, *Handbook of Metaheuristics*, p. 363–397. Springer US, Boston, MA. ISBN 978-1-4419-1665-5. URL https://doi.org/10.1007/978-1-4419-1665-5_12.
- Mladenović, N. e Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11):1097–1100.
- Nowicki, E. e Smutnicki, C. (1996). A fast taboo search algorithm for the job shop problem. *Management Science*, 42(6):797–813.
- Pekny, J. F. e Miller, D. L. (1994). A staged primal-dual algorithm for finding a minimum cost perfect two-matching in an undirected graph. *ORSA Journal on Computing*, 6(1):68–81.
- Prandtstetter, M. e Raidl, G. R. (2008). An integer linear programming approach and a hybrid variable neighborhood search for the car sequencing problem. *European Journal of Operational Research*, 191(3):1004–1022.
- Reinelt, G. (1991). TSPLIB – a traveling salesman problem library. *ORSA Journal on Computing*, 3(4):376–384.
- Solnon, C., Cung, V. D., Nguyen, A., e Artigues, C. (2008). The car sequencing problem: Overview of state-of-the-art methods and industrial case-study of the ROADEF’2005 challenge problem. *European Journal of Operational Research*, 191(3):912–927.
- Voudouris, C. (1997). *Guided local search for combinatorial optimisation problems*. PhD thesis, University of Essex.